

Basics of Classes and Objects in Python

1. What is a Class?

- A **class** is a blueprint or template for creating objects.
- It defines attributes (variables) and methods (functions) that describe the behavior and state of its objects.

2. What is an Object?

- An **object** is an instance of a class.
- It has its own unique data stored in attributes, and can use the methods defined by its class.

3. Defining a Simple Class

```
class Person:
    def __init__(self, name, age):
        self.name = name    # attribute
        self.age = age      # attribute

    def greet(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")
```

- `__init__` is a special method called a *constructor*.
- `self` refers to the current object (instance).

4. Creating Objects

```
person1 = Person("Alice", 30)
person2 = Person("Bob", 25)

person1.greet()    # Output: Hello, my name is Alice and I am 30 years old.
person2.greet()    # Output: Hello, my name is Bob and I am 25 years old.
```

5. Basic Concepts

- **Attributes:** variables that belong to the object (e.g., `name`, `age`).
- **Methods:** functions defined inside the class that operate on the object (e.g., `greet()`).
- **Constructor (`__init__`):** initializes new objects with default or given values.
- **Self:** reference to the current instance, used to access attributes and methods.

6. Example: Class with Method to Change Attribute

```
class Car:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year

    def update_year(self, new_year):
        self.year = new_year

car1 = Car("Toyota", 2020)
print(car1.year)      # Output: 2020

car1.update_year(2023)
print(car1.year)      # Output: 2023
```

7. Summary

- Classes are used to create user-defined data structures.
- Objects are concrete instances of classes.
- The `self` keyword allows each object to keep track of its own data.
- The constructor method (`__init__`) sets up the initial state of new objects.